

GLSL Shader Examples

This document contains two simple GLSL shader setups:

1. Orthographic Projection Shader
2. Perspective Projection Shader

Each setup contains:

- Vertex shader
 - Fragment shader
 - Required uniforms
 - Minimal transformation pipeline
-

1. Orthographic Projection Shader

Orthographic projection removes perspective distortion. Objects retain the same size regardless of depth.

Common uses:

- UI rendering
 - 2D games
 - Debug rendering
 - CAD-style rendering
 - Sprite systems
-

orthographic.vert

```
#version 330 core

layout(location = 0) in vec3 aPosition;
layout(location = 1) in vec3 aColor;

out vec3 vColor;

uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;

void main()
{
    vColor = aColor;
```

```
gl_Position = uProjection * uView * uModel * vec4(aPosition, 1.0);  
}
```

orthographic.frag

```
#version 330 core  
  
in vec3 vColor;  
  
out vec4 FragColor;  
  
void main()  
{  
    FragColor = vec4(vColor, 1.0);  
}
```

Example Orthographic Projection Matrix (OpenGL)

```
glm::mat4 projection = glm::ortho(  
    -10.0f,  
    10.0f,  
    -10.0f,  
    10.0f,  
    0.1f,  
    100.0f  
);
```

2. Perspective Projection Shader

Perspective projection simulates realistic depth. Objects farther away appear smaller.

Common uses:

- 3D games
 - First-person cameras
 - Third-person cameras
 - Simulation engines
 - Realistic rendering
-

perspective.vert

```
#version 330 core

layout(location = 0) in vec3 aPosition;
layout(location = 1) in vec3 aColor;

out vec3 vColor;

uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;

void main()
{
    vColor = aColor;

    gl_Position = uProjection * uView * uModel * vec4(aPosition, 1.0);
}
```

perspective.frag

```
#version 330 core

in vec3 vColor;

out vec4 FragColor;

void main()
{
    FragColor = vec4(vColor, 1.0);
}
```

Example Perspective Projection Matrix (OpenGL)

```
glm::mat4 projection = glm::perspective(
    glm::radians(45.0f),
    1280.0f / 720.0f,
    0.1f,
    100.0f
);
```

Key Difference Between Orthographic and Perspective

Orthographic

Parallel lines remain parallel.
Object size does not change with depth.

Transformation:

No perspective divide effect.

Perspective

Distant objects appear smaller.
Parallel lines converge visually.

Transformation:

Perspective divide creates depth effect.

Typical Camera Setup

Orthographic Camera

```
glm::mat4 view = glm::lookAt(  
    glm::vec3(0.0f, 0.0f, 5.0f),  
    glm::vec3(0.0f, 0.0f, 0.0f),  
    glm::vec3(0.0f, 1.0f, 0.0f)  
);
```

Perspective Camera

```
glm::mat4 view = glm::lookAt(  
    glm::vec3(0.0f, 2.0f, 5.0f),  
    glm::vec3(0.0f, 0.0f, 0.0f),  
    glm::vec3(0.0f, 1.0f, 0.0f)
```

```
glm::vec3(0.0f, 1.0f, 0.0f)
);
```

Recommended Engine Structure

A common rendering pipeline:

```
Vertex Data
  ↓
Model Matrix
  ↓
View Matrix
  ↓
Projection Matrix
  ↓
Clip Space
  ↓
Rasterization
  ↓
Fragment Shader
```

Suggested File Layout

```
/shaders
  orthographic.vert
  orthographic.frag
  perspective.vert
  perspective.frag
```

Notes

The actual difference between orthographic and perspective rendering is not inside the shader logic itself.

The difference comes from the projection matrix uploaded to:

```
uniform mat4 uProjection;
```

The shader remains mostly identical. Only the matrix contents change.

Minimal Uniform Upload Example

```
shader.Bind();

shader.SetMat4("uModel", model);
shader.SetMat4("uView", view);
shader.SetMat4("uProjection", projection);
```

Summary

Orthographic projection:

- No depth scaling
- Ideal for 2D
- Cleaner for UI
- Parallel geometry preserved

Perspective projection:

- Realistic depth
- Essential for 3D worlds
- Distance-based scaling
- Camera realism